

The background of the slide is a light blue-grey color with a complex network diagram. The diagram consists of numerous small black dots (nodes) connected by thin grey lines (edges). Some nodes are highlighted with a white circle and a black border. The network is distributed across the entire slide, with a higher density of nodes and connections in the upper and lower portions, framing the central text area.

Sistemas Distribuidos 2027-1 • Grupo 2 • Sesión 3

Anatomía de un sistema real

Cómo se hablan los servicios y las piezas que los rodean

Sábado 5 de septiembre de 2026 • Prof. Oscar Ruiz • FI-UNAM

El mapa de la sesión: 90 minutos, una decisión al final

0

La pregunta de la semana

Waldo 1994

5 min

A

La red no es una llamada local

timeout, semánticas, exactly-once

17 min

B

REST e idempotencia

Richardson, RFC 9110, Idempotency-Key

17 min

D

Demo: el sistema por fuera

caja negra

8 min

C

Las seis piezas

gateway a observabilidad

19 min

E

Los datos

políglota, familias

4 min

F

La decisión de hoy

FarmaYa

12 min

G

Para la S4

lectura y sistema en su máquina

2 min

¿Cuál de las cuatro diferencias no se puede esconder?

vota en el chat



Waldo, Wyant, Wollrath y Kendall (Sun, 1994): tratar objetos remotos como locales falla por cuatro diferencias. Tres se mitigan con diseño; una, no.

A



Latencia

la red es 4 o 5 órdenes de magnitud más lenta

B



Acceso a memoria

los punteros no cruzan; los datos se serializan

C



Concurrencia

muchos clientes tocan el mismo estado sin coordinador

D



Fallo parcial

una parte falla mientras el resto sigue, y nadie sabe qué

Después: las dos preguntas más votadas de la Discussion #2.

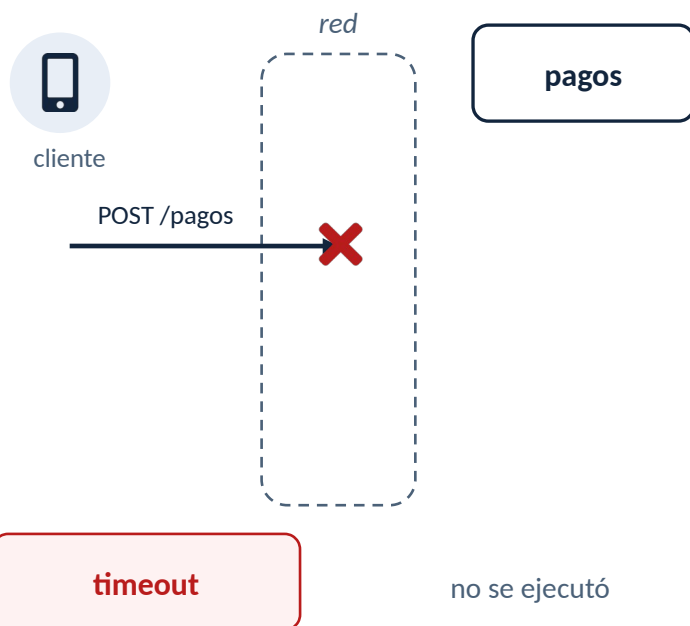
Una llamada local promete tres cosas; la red rompe las tres



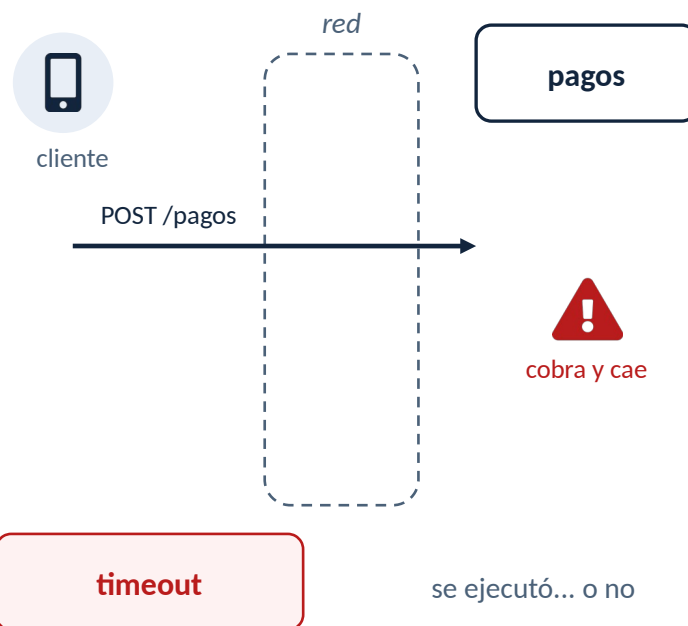
Waldo (1994): la diferencia que no se esconde es la segunda; en un solo proceso no existe "a medias".

Un timeout no dice qué pasó: tres historias idénticas para el cliente

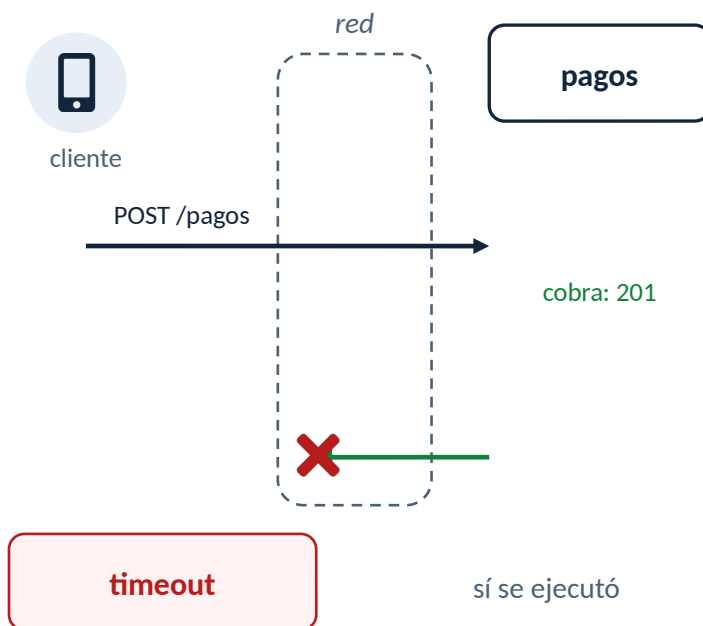
1 • La petición se pierde



2 • El servidor muere a medias



3 • La respuesta se pierde



El cliente ve exactamente lo mismo en las tres: un timeout. Asumir "no se ejecutó" es la causa clásica del cobro doble.

Predice: un cliente con timeout de 2 s contra un servidor que tarda 5 s

predice



```
HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(1)).build();
HttpRequest req = HttpRequest.newBuilder(URI.create(url)) // :8081/cobrar
    .timeout(Duration.ofSeconds(2))
    .POST(BodyPublishers.ofString("250")).build();
long t0 = System.currentTimeMillis();
try {
    var r = client.send(req, BodyHandlers.ofString());
    System.out.println("respuesta " + r.statusCode() + ": " + r.body());
} catch (HttpTimeoutException e) {
    System.out.println(e + " a los " + (System.currentTimeMillis()-t0) + " ms");
}
```



Dos preguntas

1. ¿Qué imprime el cliente y a los cuántos milisegundos?
2. ¿El servidor termina el cobro de todos modos?

[demo en vivo: codigo/TimeoutDemo](#)

El servidor (Servidor.java, puerto 8081) duerme 5 s y luego imprime "cobro hecho".

El timeout limita la espera, no el efecto

Terminal del cliente

```
$ java Cliente  
java.net.http.HttpTimeoutException:  
    request timed out a los 2015 ms  
[cliente] no se si el servidor cobro
```

2 015 ms

el cliente dejó de esperar

Terminal del servidor

```
$ java Servidor  
[servidor] peticion recibida, cobrando $250...  
[servidor] cobro hecho (5000 ms despues)  
[servidor] no pude entregar la respuesta
```

5 000 ms

el servidor cobró igual

?

lo que sabe el cliente

Regla del curso: timeouts cortos y explícitos en cada llamada (conexión y respuesta), porque un hilo bloqueado deja de atender a todos los demás. AWS Kinesis, 25-nov-2020: 17 horas de cascada por hilos agotados.

Reintentar es elegir una semántica de invocación

Semántica	Qué hace el emisor	Riesgo	Dónde se usa
At-most-once · a lo sumo una vez	Envía una vez; si hay timeout, no reintentar	Puede no ocurrir	Telemetría, notificaciones
At-least-once · al menos una vez	Reintenta hasta recibir confirmación (ack)	Puede ejecutarse varias veces	Trabajo que no puede perderse
Exactly-once · efecto de una vez	Reintenta, y el receptor deduplica por identificador	Exige un almacén durable de identificadores	Cobros, pedidos, efectos externos

exactly-once (efecto) = at-least-once + idempotencia o deduplicación



Por qué no existe la entrega exactamente una vez

Dos generales que solo se hablan por mensajeros que pueden ser capturados nunca alcanzan certeza: el último mensaje siempre puede perderse.

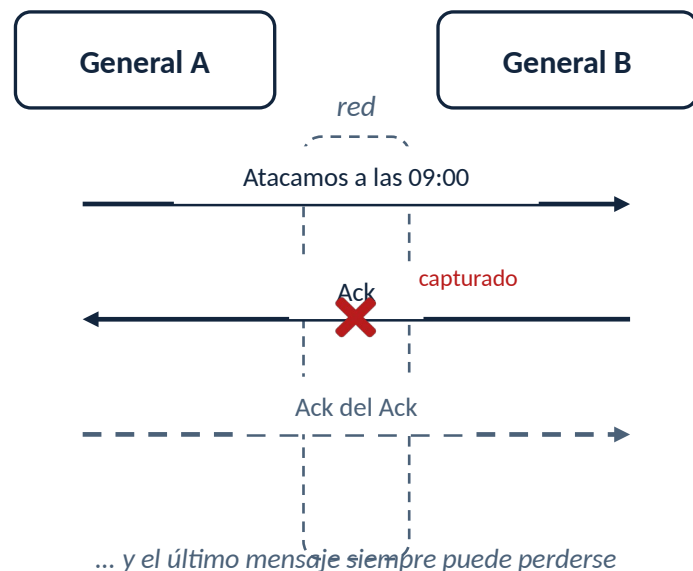


Lo que sí garantiza cada capa

TCP entrega bytes, no efectos. Kafka garantiza exactly-once solo dentro de Kafka; el efecto en su base de datos sigue siendo suyo.

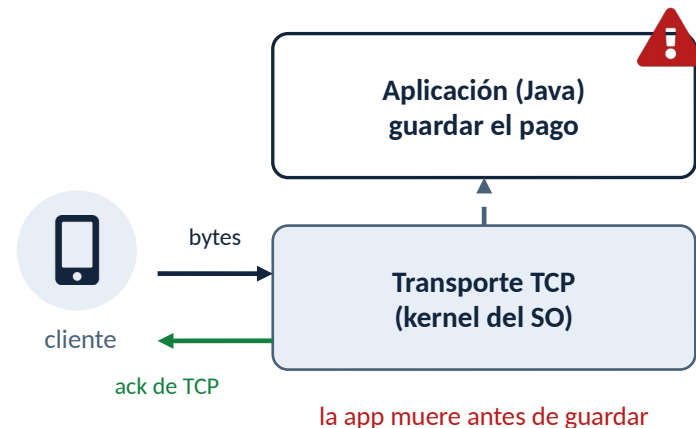
Exactly-once no existe en la red: tres razones de ingeniería

1 • La matemática: los Dos Generales



Ninguna cadena finita de confirmaciones da certeza: entregar una sola vez con total certeza es imposible.

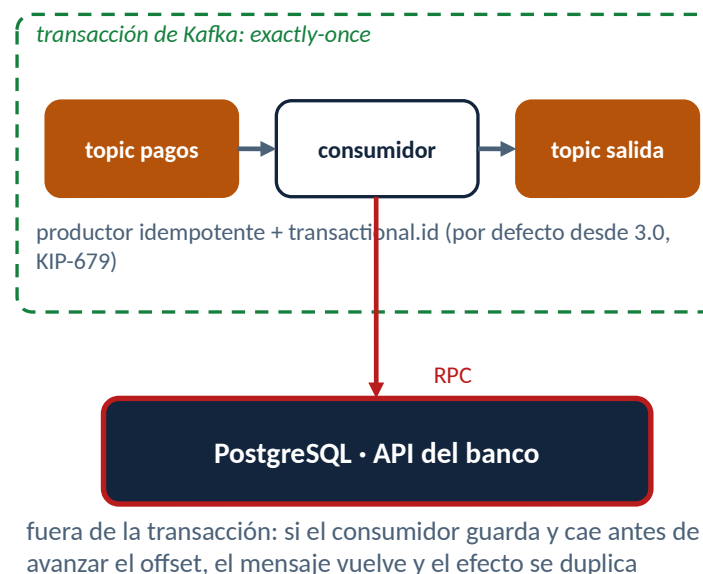
2 • Las capas: la trampa de TCP



El kernel confirma "llegaron los bytes al puerto" antes de que tu código haga nada.

TCP entrega bytes, no efectos: el ack de transporte no dice que la aplicación procesó.

3 • El alcance: la frontera de Kafka



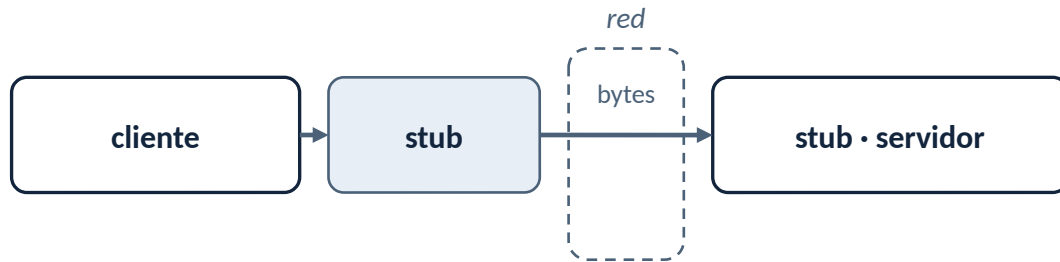
La garantía termina en la frontera de Kafka; afuera, tu receptor tiene que ser idempotente.

La entrega exactamente una vez no existe. El efecto exactamente una vez se construye: reintentos + receptor idempotente.

RPC esconde la red; REST la muestra

RPC • Birrell y Nelson, Xerox PARC, 1984

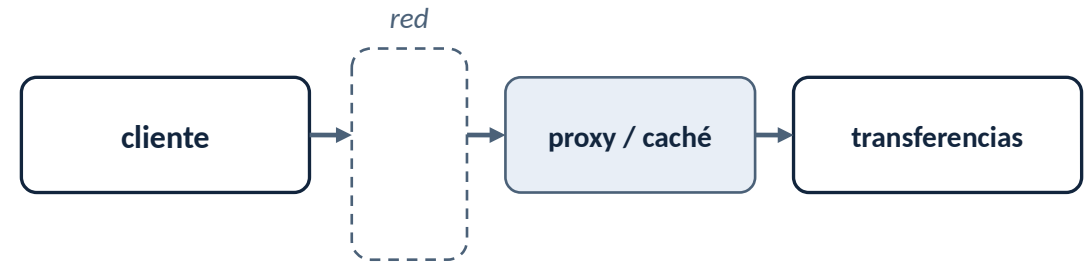
```
Recibo r = pagos.procesar(tarjeta, 250);
```



El stub finge ser el servicio: empaqueta argumentos (Lupine generaba el código; Grapevine resolvía a quién llamar). Costo: la red desaparece del código y con ella el timeout.

REST • Fielding, tesis doctoral, 2000

```
POST /transferencias {"de":42,"a":7,"monto":250}  
201 Created Location: /transferencias/8112
```



Recursos con nombre, un vocabulario fijo de verbos y códigos que toda la infraestructura entiende sin conocer el negocio: la dirección va en el sobre.

Mismo problema (fallo parcial), dos filosofías: ocultar la red o hacerla explícita. La segunda gana el tráfico público; la primera vuelve en la S4 con gRPC.

"Uso POST para todo y devuelvo JSON, ¿eso es REST?" Los cuatro niveles de Richardson

Nivel 0 • el pantano de POX

esto no es REST

HTTP como túnel: todo por POST a un único endpoint. Es RPC disfrazado.

```
POST /api {"accion":"cancelarPedido","id":8112}
```

Nivel 1 • recursos

Una URI por entidad; los verbos todavía no importan.

```
POST /pedidos/8112/cancelar
```

Nivel 2 • verbos y códigos con rigor

aquí vive la industria

GET, POST, PUT, DELETE y 200, 201, 404, 409 con su semántica. El estándar real de la industria.

```
DELETE /pedidos/8112 → 204 No Content
```

Nivel 3 • HATEOAS

La respuesta trae los enlaces de lo que puedes hacer después. El menos adoptado en la práctica.

```
{"estado":"pagado","_links":{"cancelar":"/pedidos/8112/cancelacion"}}
```



Adelanto de la S4: RPC regresó como gRPC

REST domina el tráfico público. Para el tráfico interno entre microservicios, RPC volvió con fuerza como gRPC (Google, 2015).

Aprendió la lección de 1984: cada llamada lleva un deadline obligatorio; si se vence, DEADLINE_EXCEEDED, nada de esperar para siempre.



Leonard Richardson (2008), popularizado por Fowler (2010): Fielding solo reconoce como REST el nivel 3.

Idempotencia: hacerlo dos veces deja lo mismo que hacerlo una vez

$$f(f(x)) = f(x)$$

Una operación es idempotente si repetirla no cambia el estado más allá de la primera vez. Importa porque reintentar es repetir.

 Prueba rápida

Ejecútala dos veces en tu cabeza. ¿El estado final es el mismo? Entonces se puede reintentar a ciegas.

 Dos confusiones

No es "misma respuesta": DELETE da 204 y luego 404, mismo efecto.
No es "inofensivo": DELETE borra y es idempotente.

Idempotente: repetir no cambia más	No idempotente: repetir acumula
Botón del elevador: cinco veces, un elevador	Apagador tipo toggle: dos veces, vuelve a apagado
saldo = 500 (asignación absoluta)	saldo += 100 (incremento)
PUT /cuentas/42 {"saldo": 500}	POST /cuentas/42/abonos {"monto": 100}
DELETE /sesiones/k1 (204 y luego 404)	Enviar el correo de bienvenida (dos correos)
Marcar el recibo 8112 como pagado	Crear un pedido (dos pedidos)
Registrar el pago con clave k-71 (si ya existe, no pasa nada)	Registrar el pago sin clave (cada llamada cobra)

La última fila es el truco de toda la sesión: cualquier operación se vuelve idempotente si le pones un identificador y recuerdas cuáles ya viste.

Seguro no es lo mismo que idempotente (RFC 9110)

Método	Seguro: solo lectura	Idempotente: repetir no cambia más	Reintento automático permitido
GET · HEAD · OPTIONS	sí	sí	sí
PUT (reemplazo completo)	no	sí	sí
DELETE	no	sí	sí
POST (crear, cobrar)	no	no	solo con Idempotency-Key
QUERY (consulta con cuerpo, RFC 10008, jun-2026)	sí	sí	sí



La respuesta puede cambiar y seguir siendo idempotente

DELETE /pedidos/8112 devuelve 204 la primera vez y 404 la segunda. El efecto en el servidor es el mismo: el pedido no existe. Idempotencia habla del efecto, no del código de estado.



Lo que dice el RFC sobre reintentar

"A client SHOULD NOT automatically retry a request with a non-idempotent method unless it has some means of knowing that the request semantics are idempotent". Ese medio es la clave de idempotencia.

Vota: ¿cuáles se pueden reintentar a ciegas tras un timeout?

vota en el chat



A

PUT /cuentas/42

```
{ "saldo": 500 }
```

B

POST /cuentas/42/abonos

```
{ "monto": 100 }
```

C

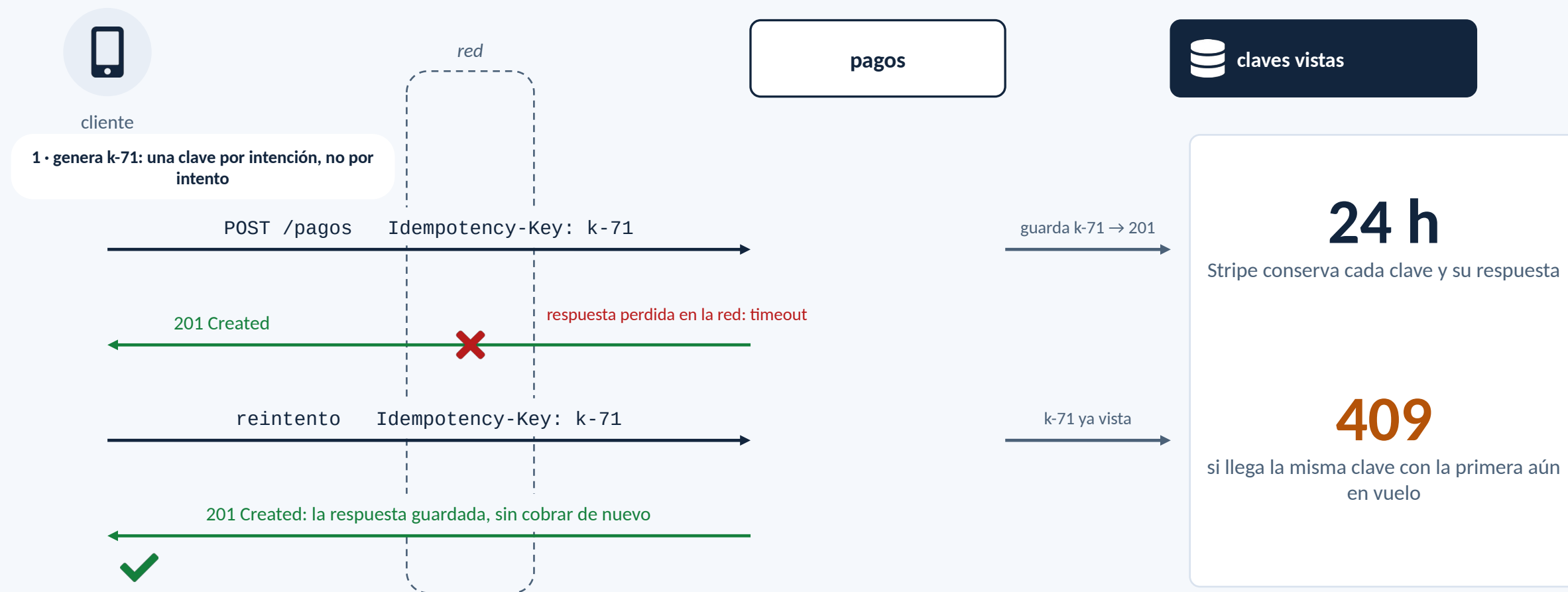
DELETE /sesiones/k1

D

GET /cuentas/42

Escriban las letras. Pista: pregúntense qué queda en la base de datos si la operación se ejecuta dos veces.

Idempotency-Key: una intención, una sola ejecución



Patrón popularizado por Stripe (2017). La clave viaja en una cabecera HTTP; el servidor guarda código y cuerpo de la primera respuesta, incluso si fue error.

Un Map en memoria no basta: la carrera TOCTOU y el UNIQUE

Comprobar y luego actuar (Map en memoria)

t = 0 ms petición 1: ¿visto(k-71)? no → cobra → marca

t = 5 ms petición 2: ¿visto(k-71)? todavía no → cobra → marca

2 cobros

y con dos réplicas, dos mapas que no se ven

Insertar es comprobar y reservar (tabla con UNIQUE)

t = 0 ms INSERT k-71 → ok → cobra → guarda respuesta

t = 5 ms INSERT k-71 → falla (UNIQUE) → devuelve la guardada

1 cobro

la base de datos serializa; funciona con N réplicas

```
@Column(unique = true) String clave;                                // en la entidad IdempotencyRecord
@Transactional public Registro procesarNuevo(String clave, PagoRequest req) {
    var r = repo.saveAndFlush(new Registro(clave));                  // INSERT ahora: clave repetida = excepcion
    r.guardar(pasarela.cobrar(req)); return r; }                     // el efecto ocurre UNA vez
catch (DataIntegrityViolationException dup) {                      // en el controlador
    return respuestaGuardada(clave); }                               // misma respuesta, sin cobrar
```


Demo: el sistema de referencia, por fuera

demo en vivo



Pieza	Rol	Puerto
API Gateway · Spring Cloud Gateway	entrada única	8080
Discovery · Eureka	registro de servicios	8761
Config server · Spring Cloud Config	configuración central	8888
Autenticación · Keycloak	quién es y qué puede hacer	8180
Broker · RabbitMQ	mensajería asíncrona	5672 · consola 15672
Caché · Redis	lecturas rápidas en memoria	6379
Datos · PostgreSQL · MongoDB	almacenes de los servicios	5432 · 27017
Observabilidad · Grafana · Prometheus · Jaeger	métricas y trazas	3000 · 9090 · 16686

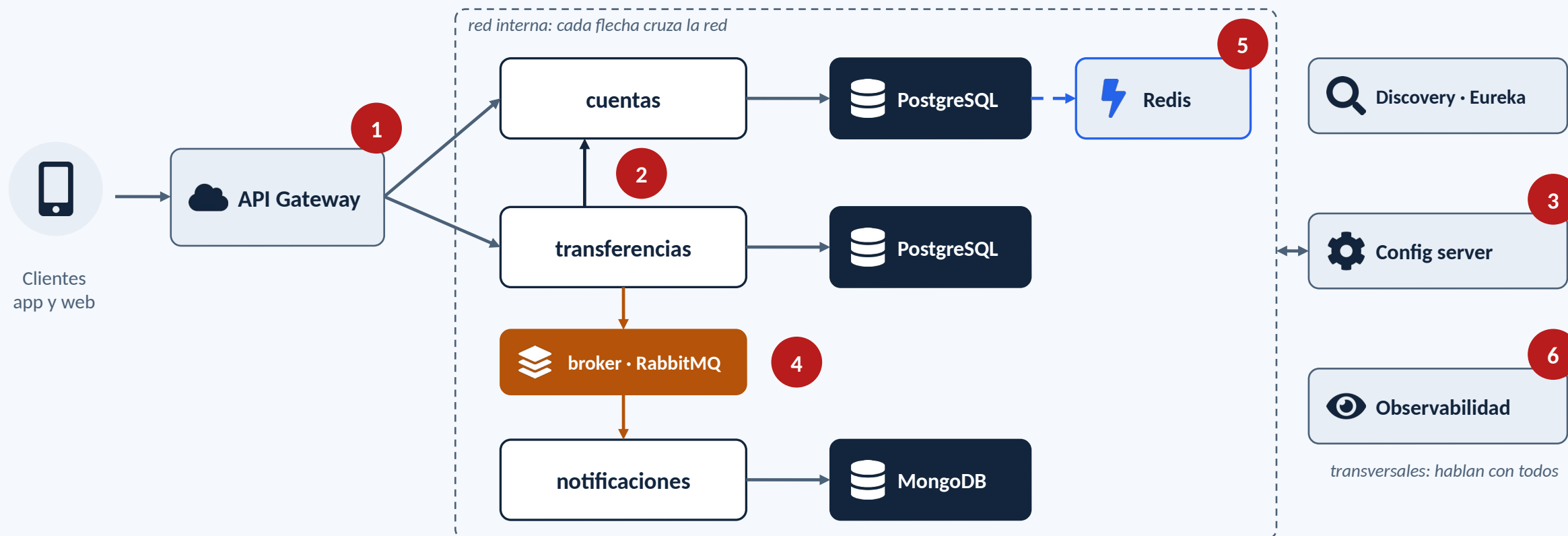
Puertos por defecto; los definitivos van en el compose del curso (S4).

```
$ docker compose up -d
$ docker compose ps
$ docker compose logs -f gateway
```

>_ Qué van a ver

Los mismos comandos del lab de hoy, solo que ahora son ~12 contenedores coordinados. Anoten qué puerto abre cada pieza: la próxima semana lo levantan ustedes.

Partir el monolito crea cinco problemas nuevos y uno de visibilidad



1 · ¿Por dónde entra el cliente si ahora son N servicios?

3 · ¿De dónde sale la configuración de cada uno?

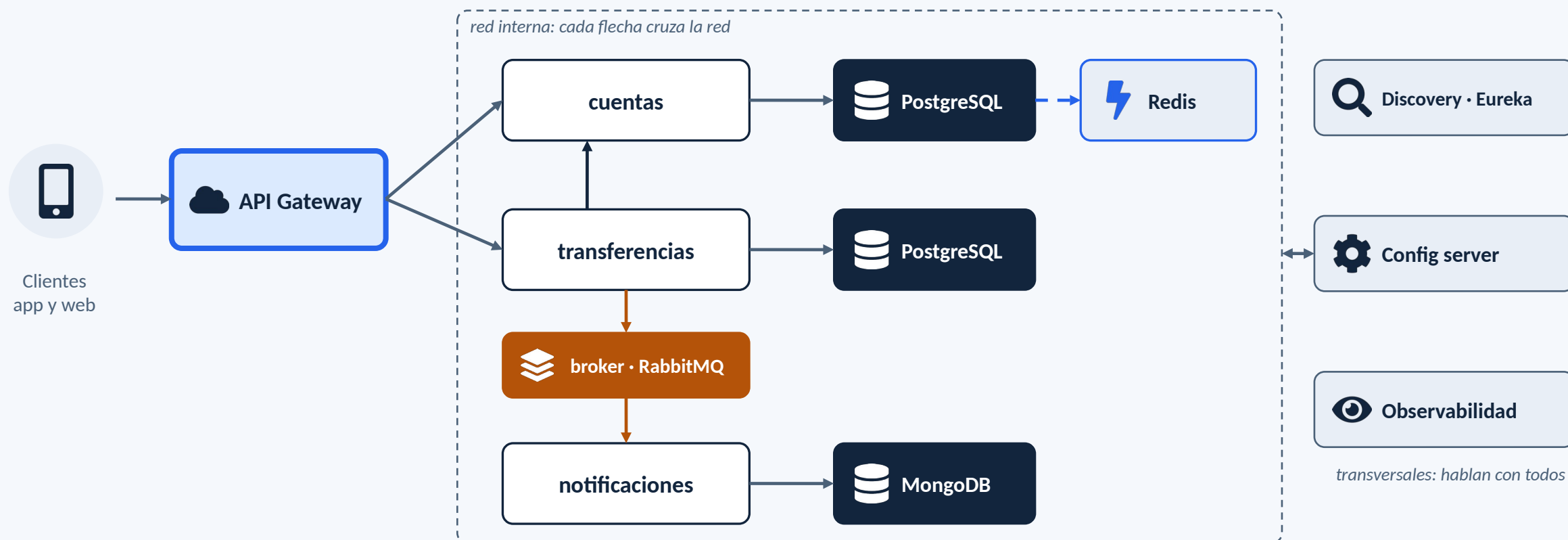
5 · ¿Cómo evitar leer lo mismo mil veces de la base?

2 · ¿Cómo se encuentran, si las direcciones cambian?

4 · ¿Qué pasa con la operación si el otro está caído?

6 · ¿Dónde falló una operación que cruzó varios?

API Gateway: una sola puerta, políticas una sola vez



Problema que resuelve

Sin gateway cada servicio expone su puerto: TLS, identidad y límites se programan N veces y el cliente se acopla a la topología.



Dato duro

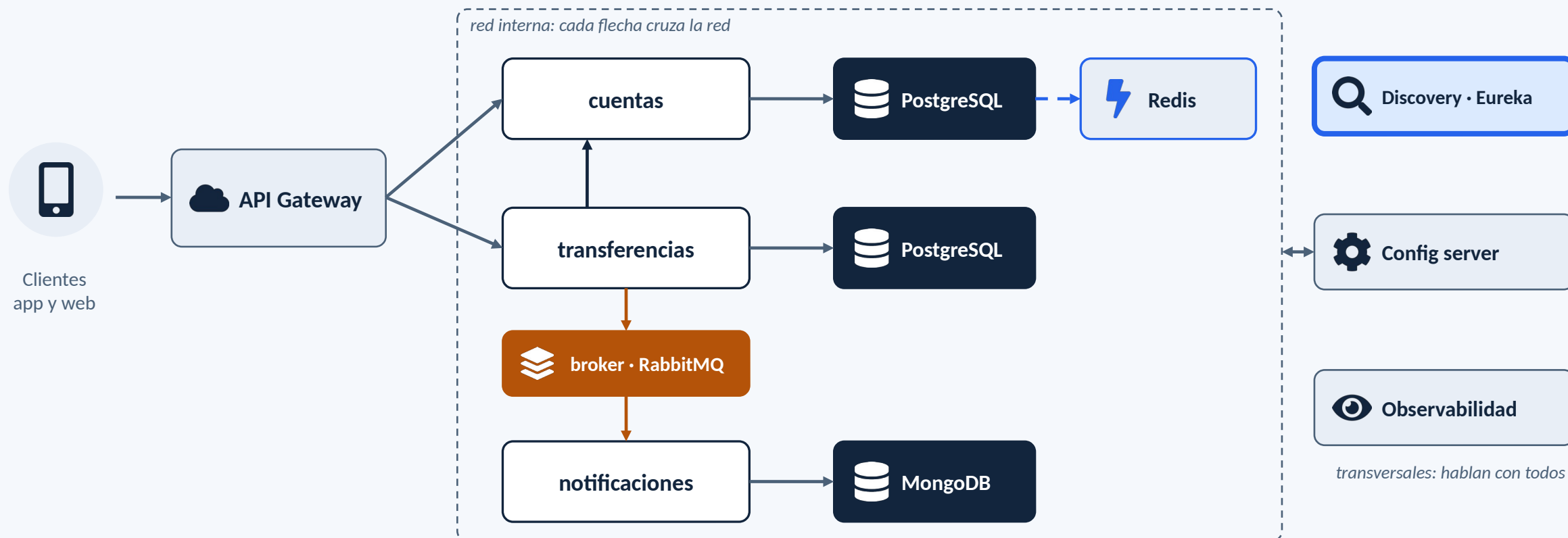
Spring Cloud Gateway 5 (nov-2025). Kubernetes Gateway API es GA desde el 31-oct-2023; Ingress quedó congelado.



Incidente real

Cloudflare, 18-nov-2025: un archivo creció más de lo previsto, el proxy central hizo panic y devolvió 5xx de 11:28 a 17:06 UTC.

Service discovery: el nombre es estable, la dirección no



Problema que resuelve

En contenedores la IP cambia en cada arranque y al escalar hay N instancias: el nombre (naming) se separa de la dirección (binding).



Dato duro

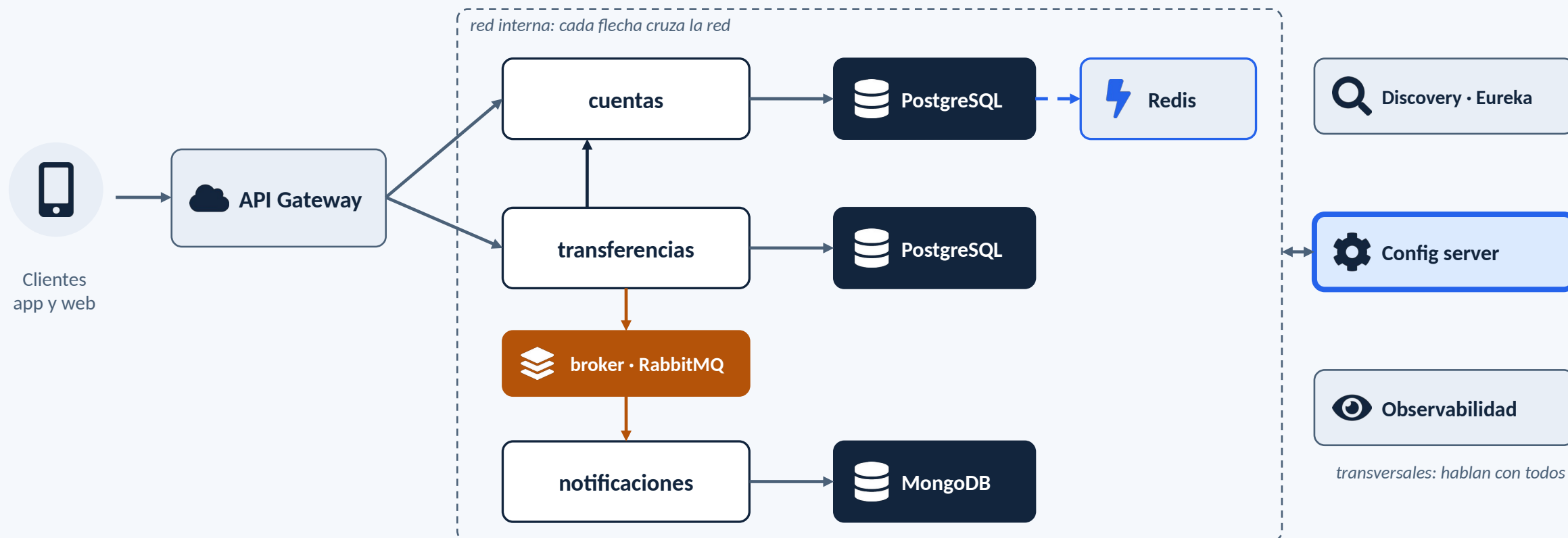
Eureka: heartbeat cada 30 s, expulsión a los 90 s. En Kubernetes lo hacen Service + CoreDNS. La JVM cachea DNS 30 s.



Incidente real

Roblox, 28 al 31-oct-2021: Consul degeneró bajo carga, los servicios dejaron de encontrarse y la telemetría dependía de Consul. 73 horas.

Config server: la imagen es el uniforme, la configuración es el gafete



? Problema que resuelve

La misma imagen corre en desarrollo y producción con valores distintos; config dentro de la imagen = reconstruir para cambiar un valor.

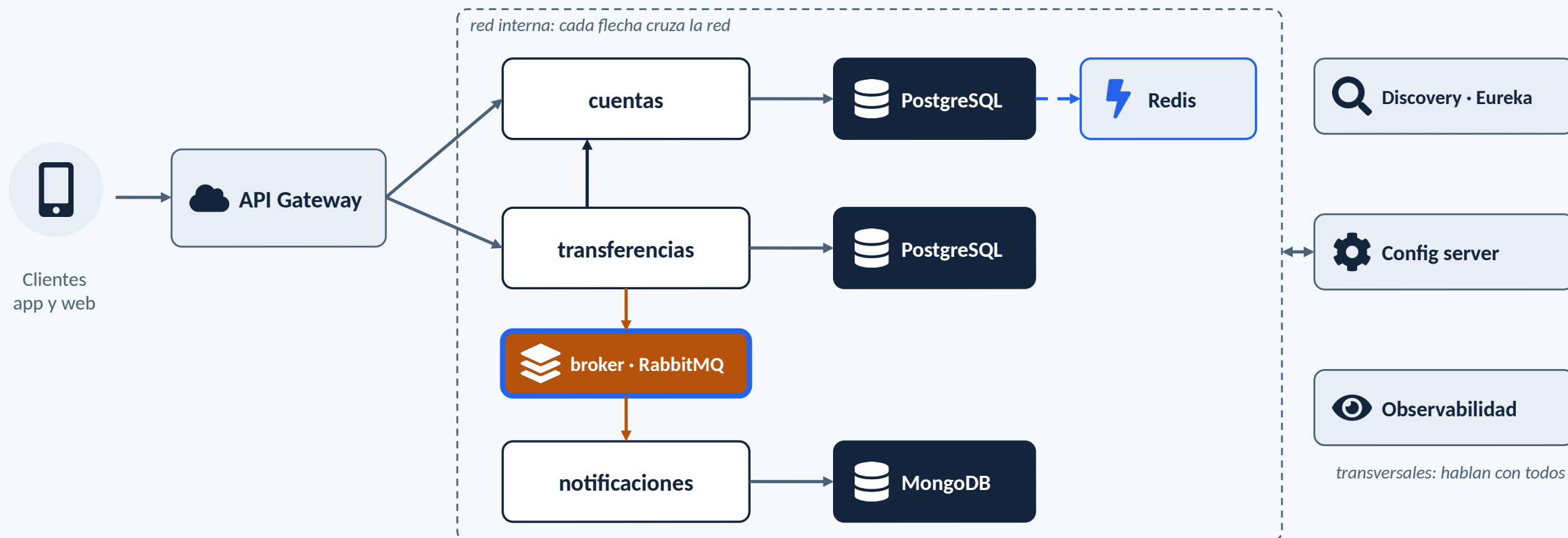
📖 Dato duro

Factor III de The Twelve-Factor App: config es "everything that is likely to vary between deploys". Spring Cloud Config 5, ConfigMaps, Vault.

⚠️ Incidente real

Fastly, 8-jun-2021: un cambio de configuración válido disparó un bug latente; 85 % de la red con errores durante 49 minutos.

Broker: de "procesar ahora" a "procesar eventualmente"



Problema que resuelve

Una llamada síncrona acopla disponibilidad: si notificaciones está caído, la transferencia falla. El broker guarda el mensaje para después.



Dato duro

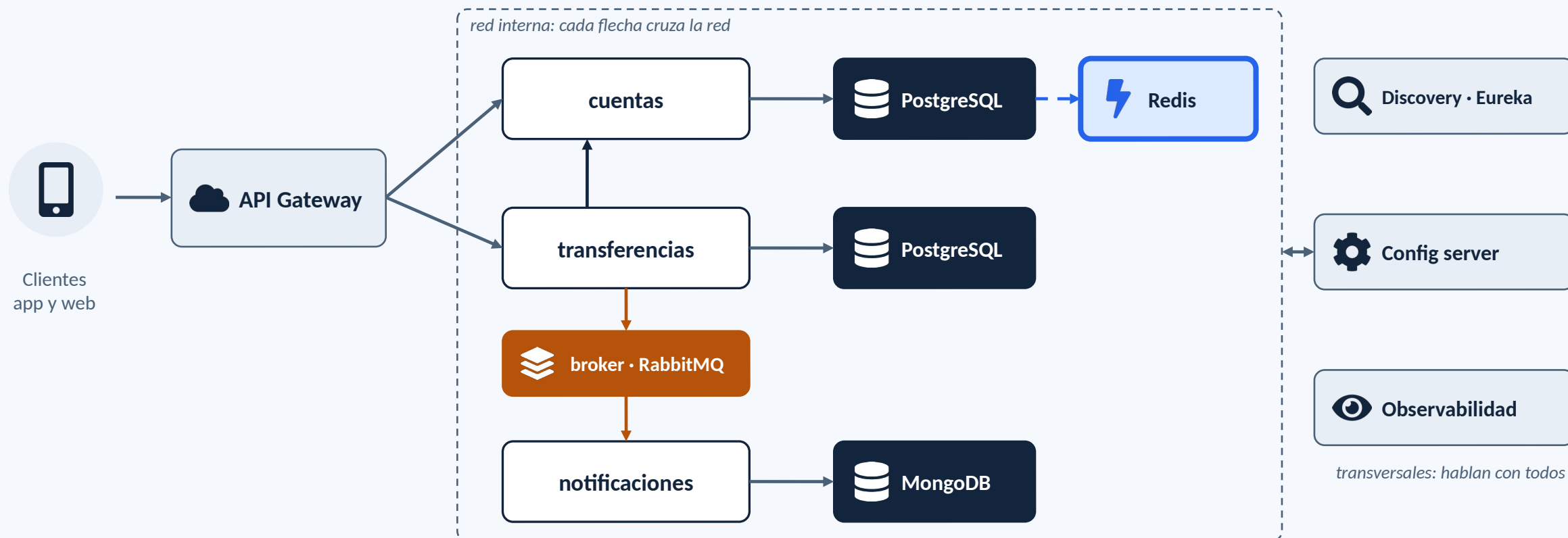
At-least-once: ack tras persistir; sin ack, reentrega; N fallos, dead-letter queue. RabbitMQ 4.0 (sep-2024), Kafka 4.0 sin ZooKeeper (mar-2025).



Incidente real

AWS Kinesis, 25-nov-2020: Cognito escribía al broker de forma bloqueante; con el broker caído, el servicio "asíncrono" cayó también.

Caché: una copia que se permite estar vieja



Problema que resuelve

La base de datos es el recurso caro y compartido. Cache-aside: si la llave está, hit; si no, miss: se lee de la base y se guarda con TTL.



Dato duro

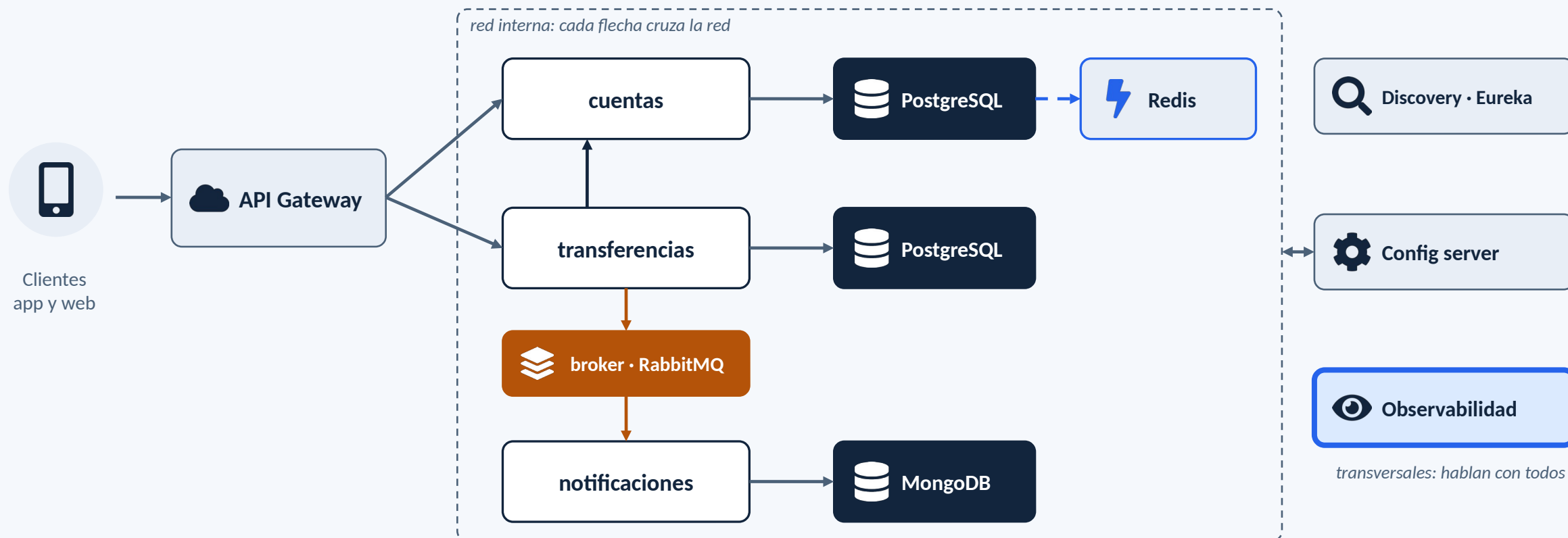
De 99 % a 98 % de aciertos se duplica el tráfico a la base. Lo difícil es invalidar. Redis 8 (may-2025, AGPLv3) y Valkey (2024).



Incidente real

Slack, 22-feb-2022: nodos de Memcached sustituidos por instancias vacías en el pico; la base hizo timeout y la caché no se podía rellenar.

Observabilidad: sin trace id hay veinte bitácoras y ninguna historia



? Problema que resuelve

Cuando una operación cruza varios servicios, ¿dónde falló y dónde tardó? Logs (qué pasó aquí), métricas (cómo se comporta), trazas (el recorrido).

📖 Dato duro

W3C traceparent = versión, trace id, parent id, flags; viaja en cada llamada. OpenTelemetry con las tres señales estables; Jaeger v2 (nov-2024).

⚠ Incidente real

El monitoreo que depende de lo que monitorea: la telemetría de Roblox dependía de Consul; CloudWatch dependía de Kinesis (2020).

Diseña en 3 minutos: notificaciones está caído

en parejas



El reto

Mexi Banco: una transferencia de \$250 debe descontar de cuentas, registrarse en transferencias y avisar por SMS. El servicio de notificaciones lleva 10 minutos caído.

Dibujen el flujo con las piezas de hoy y respondan: ¿la transferencia se completa? ¿Cuándo llega el SMS? ¿Qué pieza guarda el aviso mientras tanto y qué pasa si llega dos veces?



Criterio de éxito

Aparecen el broker, el ack y la idempotencia del consumidor.

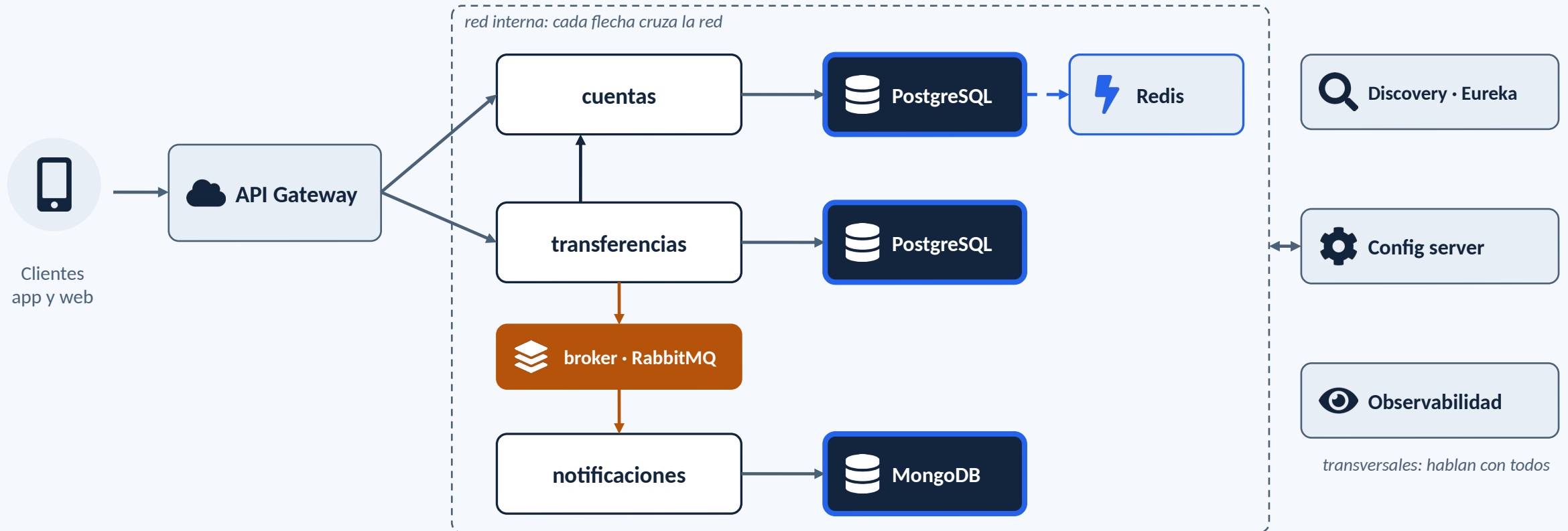
Se dice qué se sacrifica: el SMS deja de ser inmediato.

Dos parejas comparten su dibujo en el chat.

3 min

reloj en pantalla

Cada servicio es dueño de su almacén; se elige por patrón de acceso



La regla

Los datos de un servicio se leen por su API o sus eventos, nunca entrando a la base de datos ajena.



Dato duro

Fowler acuñó polyglot persistence el 16-nov-2011: primero cómo se manipulan los datos, después la tecnología.



Lo que se pierde

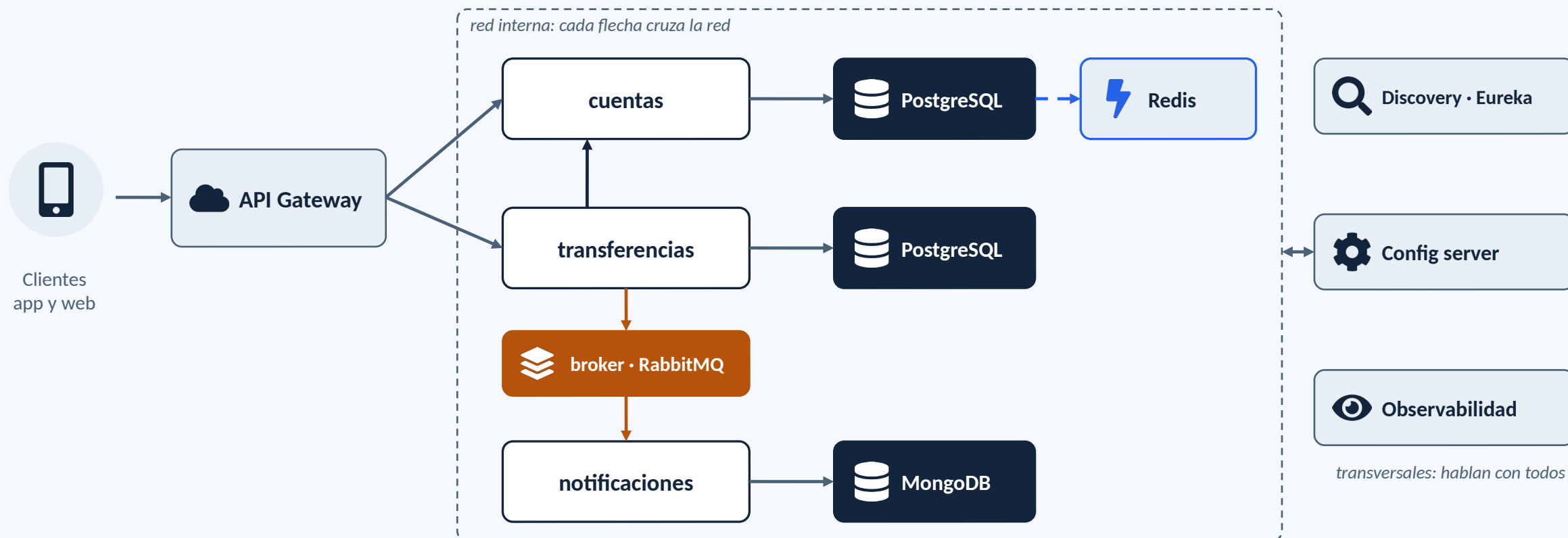
No hay joins ni transacciones entre servicios: la consistencia se vuelve diseño (S6, S12); hacen falta eventos e idempotencia.

Seis familias, seis preguntas: el criterio es cómo se lee y cómo se escribe

Familia	Fortaleza	Caso típico	Ejemplo · AWS
Relacional	ACID, joins, integridad (UNIQUE)	cuentas, transferencias, libro mayor	PostgreSQL · Aurora, RDS
Documental	el agregado completo en una lectura, esquema flexible	expediente de cliente, catálogo	MongoDB · DocumentDB
Llave-valor	acceso por llave en microsegundos	caché, sesiones, contadores	Redis, Valkey · DynamoDB, ElastiCache
Columnar	agregaciones sobre muchas filas y pocas columnas	analítica, series de tiempo	Cassandra · Redshift, Keyspaces
Grafo	recorrer relaciones de profundidad variable	fraude, recomendaciones, avales	Neo4j · Neptune
Objetos	archivos grandes e inmutables por URL	documentos, imágenes, respaldos	MinIO · S3

En el sistema de referencia conviven tres: PostgreSQL (transaccional), MongoDB (documentos) y Redis (llave-valor). Profundización en la S9: replicación, sharding, CRDTs.

El mapa completo: seis piezas, un problema cada una



Gateway: una puerta

Discovery: nombre estable

Config: fuera de la imagen

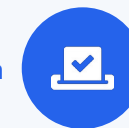
Broker: al menos una vez

Caché: copia con TTL

Observabilidad: trace id

La decisión de hoy • FarmaYa: ¿reintentar o no?

ritual • 12 min



? El caso (ajeno, ensayo, no se califica)

FarmaYa vende medicinas con entrega a domicilio. Al pagar, la app llama a POST /pedidos/{id}/pagar, que llama a la pasarela de tarjetas. La pasarela tarda de 1 a 8 s y a veces no responde. El timeout de la app es de 3 s. Se venció. ¿Qué hace la app?

A • Reintentar ya

at-least-once

Persigue: ningún pedido pagado se pierde.
Exige: Idempotency-Key en la pasarela.
Sacrifica: latencia y depender de que la pasarela deduplique de verdad.

B • No reintentar

at-most-once

Persigue: nunca cobrar dos veces.
Pide al cliente revisar su estado.
Sacrifica: pedidos en limbo y soporte manual.

C • Evento al broker

"pago solicitado" y responder "en proceso"

Persigue: la app disponible aunque la pasarela esté lenta.
Exige: consumidor idempotente y notificar el resultado.
Sacrifica: confirmación inmediata.

elección: A | B | C porque: el requisito que persigo sacrificio: lo que dejo atrás

Mismo formato de tres líneas que la Decisión 0. Los 10 equipos deciden en el chat; dos defienden en voz.

Para la S4: el sistema en su máquina, y gRPC



Lectura 3 • para la S4

The Twelve-Factor App (12factor.net), los 12 factores; atención a III Config y VI Procesos.

Burns, Grant, Oppenheimer, Brewer y Wilkes, "Borg, Omega, and Kubernetes" (2016).



Entrega

2 preguntas + 1 hallazgo en la Discussion de la semana.

Antes del jueves 10 de septiembre, 23:59.

Con esta ya cuenta para la calificación de lecturas.



S4 • 12 de septiembre

docker pull de las imágenes provistas del curso, levantar el sistema completo con el compose dado y explorarlo por dentro.

La teoría abre con gRPC y el criterio REST vs gRPC.

38 → 9 bytes

el mismo objeto en JSON y en Protobuf: el adelanto de la S4

Requisito: Docker funcionando, exactamente lo del lab de hoy

Ocho preguntas que hoy ya pueden responder

1

¿Qué tres historias esconde un timeout y cuál es la única salida segura?

2

¿Qué diferencia hay entre at-least-once y exactly-once como efecto?

3

¿Por qué DELETE es idempotente aunque responda 204 y luego 404?

4

¿Por qué un Map en memoria no basta para las claves de idempotencia?

5

¿Qué problema resuelve el API gateway y qué pasa si él falla?

6

¿Cómo encuentra un servicio a otro si las IPs cambian, y cada cuánto late Eureka?

7

¿Cuándo conviene un broker en lugar de una llamada directa, y qué se sacrifica?

8

¿Con qué criterio se elige el tipo de base de datos de cada servicio?

La que no puedan responder, esa es la que se estudia esta semana. Están respondidas en la guía.

Fuentes de la sesión

- Waldo, Wyant, Wollrath y Kendall. A Note on Distributed Computing. Sun Microsystems, SMLI TR-94-29, 1994 (la Lectura 2).
- Birrell y Nelson. Implementing Remote Procedure Calls. ACM TOCS 2(1), 1984.
- Fielding. Architectural Styles and the Design of Network-based Software Architectures, cap. 5. UC Irvine, 2000.
- RFC 9110, HTTP Semantics (IETF, 2022), §9.2.1 y §9.2.2 • RFC 10008, The HTTP QUERY Method (IETF, 2026).
- Stripe. Idempotent requests (docs.stripe.com) • Leach, Designing robust and predictable APIs with idempotency, 2017.
- AWS. Summary of the Amazon Kinesis Event in US-EAST-1, 25-nov-2020 • Brooker, Exponential Backoff And Jitter, 2015.
- Roblox, Return to Service 10/28-10/31 2021 • Cloudflare, outage on November 18, 2025 • Slack, Incident on 2-22-22 • Fastly, Summary of June 8 outage, 2021 • Twilio, Billing Incident Post-Mortem, 2013.
- Spring Boot 4.1, Spring Cloud 2025.1 (Gateway, Netflix, Config), Spring AMQP 4.0: documentación de referencia en docs.spring.io.
- Fowler. PolyglotPersistence, 2011 • Newman. Building Microservices, 2.^a ed., 2021 • Kleppmann. Designing Data-Intensive Applications, 2017.
- Sigelman et al. Dapper, Google 2010 • W3C Trace Context Level 1 • OpenTelemetry specification status.

Enlaces completos y numerados en 01-investigacion.md (103 fuentes) y en lecturas/README.md del repo del curso.